

Data and Applications

Project Phase 3: Relational Model and Normalization

Team Name: The Unnecessary SeQueL

Team Members:

G Sai Sri Harsha – 2024101034

Yakkala Viswanadha Kartikeya – 2024111033

Phanindra Gollapalli – 2024101068

Relational Model converted from ER diagram:

[Link to Pre-Normalized Relational Model](#)

Relational Model to Normalized Form:

[Link to Post-Normalized Relational Model](#)

Explanations:

ER diagram to Relational Model:

The relational schema was obtained by systematically converting all components of the ER diagram into relations following standard ER-to-Relational mapping rules.

1. Mapping Strong Entity Types:

Each strong entity in the ER model (Character, Crew, DevilFruit, Location, Poneglyph, Skill) was converted into a separate relation containing all its simple attributes.

Composite attributes such as *Name* and *Origin Location* were flattened into atomic attributes (FirstName, MiddleName, LastName, etc.).

Primary keys were directly taken from the ER diagram:

- Character (Character_ID PK)

- Crew (Crew_ID PK)
- DevilFruit(Fruit_ID PK)
- Poneglyph(Poneglyph_ID PK)
- Location (Location_ID PK)
- Skill (Skill_ID PK)

2. Mapping Subclasses (Specialization)

The ER diagram contains a disjoint specialization of Character into Pirate, Marine, and Revolutionary.

We used the Primary-Key approach, where:

- The primary key of the superclass (Character_ID) is also the primary key and foreign key in each subclass.

Thus:

- Pirate (Character_ID PK/FK)
- Marine (Character_ID PK/FK)
- Revolutionary (Character_ID PK/FK)

Each subclass relation stores only subclass-specific attributes.

3. Mapping Weak Entities

Vivre_Card

- Weak entity identified by Creator_ID
- Mapped to:
Vivre_Card(Card_ID PK, Creator_ID FK, CurrentOwner_ID FK, Condition)
- Creator_ID is UNIQUE since each character creates at most one card.

JollyRoger

- Weak entity fully dependent on Crew
- Mapped to:
Jolly_Roger(Crew_ID PK/FK, SkullDesign, Symbolism)

Colour_Scheme (multivalued attribute of Jolly Roger)

Converted into a separate relation:

Colour_Scheme(Color, Crew_ID FK, PK (Color, Crew_ID))

4. Mapping Multi-Valued Attributes

Character Haki

Haki is multi-valued, so we created a separate relation:

HAKI (Character_ID FK, Haki_Type, PK(Character_ID, Haki_Type))

This ensures each character can have multiple Haki types.

5. Mapping Relationship Types

HAS_SKILL (M: N)

Mapped to a linking table:

HAS_SKILL (Character_ID FK, Skill_ID FK, PK (Character_ID, Skill_ID))

EATS (1:1 Optional)

Mapped as a relationship relation:

EATS (Character_ID FK, Fruit_ID FK, PK (Character_ID, Fruit_ID))

Allows both sides to be optional.

HAS_CARD (M: N)

Mapped to:

HAS_CARD (Character_ID FK, Card_ID FK, PK (Character_ID, Card_ID))

IN_CREW (1: N with attribute "Role")

Mapped by placing a foreign key in Pirate plus the Role attribute:

Pirate (Character_ID PK/FK, Crew_ID FK, Role, ...)

CAPTAINS (1:1)

Mapped to its own relation because neither side has total participation:

CAPTAINS (Captain_ID FK/PK, Crew_ID FK)

CONTROLS (Crew → Island)

Mapped by storing the controlling crew ID in Island:

Island (... , Controlling_Crew_ID FK)

STATIONED_AT for Marines and Revolutionaries

Mapped through foreign keys:

Marine (... , Stationed_Island_ID FK)

Revolutionary (... , Stationed_Island_ID FK)

6. Mapping N-ary Relationships (N > 2)

Because ERDPlus cannot show N-ary diamonds, these were mapped manually:

BATTLE (ternary)

Between two characters and a location.

Mapped to relation:

BATTLE (Battle_ID PK, Faction1_ID FK, Faction2_ID FK, Location_ID FK, Casualties, Winner)

INFO_LEAK (4-ary)

Mapped to:

INFO_LEAK (Pirate_ID FK, Marine_ID FK, Poneglyph_ID FK, Location_ID FK)

Primary Key is the combination of all four FKs.

CONTEST (4-ary)

Contest always involves Crew + Marine + Location + Poneglyph:

CONTEST (Crew_ID FK, Marine_ID FK, Location_ID FK, Poneglyph_ID FK, Casualties, Winner)

Primary Key = {Crew_ID, Marine_ID, Location_ID, Poneglyph_ID}

7. Mapping Location Specialization

Location is a superclass with three subclasses: Island, Sea, Sky Island.

Used **Primary Key inheritance**:

- Island (Location_ID PK/FK, Climate, DominantSpecies, StrategicImportance)
- Sea (Location_ID PK/FK, SeaType, DangerLevel)
- SkyIsland(Location_ID PK/FK, Altitude, CloudType, Strategic Importance)

Relational Model to 1NF:

The relational model obtained from ER mapping already satisfied most 1NF requirements. However, to formally convert the schema into **First Normal Form (1NF)**, we applied the following transformations:

1. Removal of Multi-Valued Attributes

In the ER model, the only multi-valued attributes were:

- **Haki** (multi-valued for Character)
- **Colour Scheme** (multi-valued for Jolly Roger)

To satisfy 1NF (no multi-valued attributes):

Haki was placed into a separate relation

HAKI (Character_ID FK, Haki_Type, PK (Character_ID, Haki_Type))

Colour Scheme was placed into a separate relation

Colour Scheme (Crew_ID FK, Color, PK (Crew_ID, Color))

Thus, all attributes in every table now hold atomic values.

2. Flattening Composite Attributes

Composite attributes from the ER diagram were broken down into atomic attributes during mapping, which fulfills 1NF atomicity:

- **Name** → **FirstName, MiddleName, LastName, Epithet**
- **Origin Location** → **OriginIslandName, OriginSeaName**

This ensures no attribute contains structured or nested data.

3. Ensuring Every Attribute Holds a Single Value (Atomicity)

In all relations, every attribute now stores **exactly one value**, not sets or lists.

Examples:

- Role in Pirate is a single value, not a list of roles
- Fruit Type, Sea Type, Status, Condition, etc. are atomic values
- No repeating attributes like “Haki1, Haki2...” exist

Thus, **all attributes are atomic**.

4. Removing Repeating Groups

Any part of the model that could produce repeating groups was converted into separate relations:

- Skills linked via HAS_SKILL
- Poneglyph ownership via OWNS_PONEGLYPH
- Card ownership via HAS_CARD
- EATS as a separate link table

There are **no repeating groups** left in any table.

5. Handling 1: N and M: N Relationships Properly

Placing FK attributes into relationships correctly eliminates possible nested repeating values.

Examples:

- Marines stationed at islands → stored using a **single FK** StationedIslandID
- Pirates belonging to crews → stored using **Crew_ID FK**
- Owned poneglyphs → stored using a **separate linking relation**

This ensures that a single row never stores multiple values in one column.

6. Keys Properly Defined (Needed for 1NF)

Every relation now has a **well-defined primary key**, ensuring rows are uniquely identifiable.

Examples:

- Skill_ID identifies Skills
- Character_ID identifies Characters
- Composite keys used where required
(e.g., HAS_SKILL, HAS_CARD, OWNS_PONEGLYPH, HAKI, Colour_Scheme)

This satisfies the 1NF rule that each row is unique.

After decomposing multi-valued attributes, flattening composite attributes, creating separate relations for M:N relationships, and ensuring atomicity of every attribute, the relational schema now fully satisfies the requirements of First Normal Form (1NF). No attributes contain repeating groups or non-atomic values, and every relation includes a valid primary key.

For 1 NF to 2 NF:

After converting the relational schema to 1NF, we analysed all relations for partial dependencies. A relation violates 2NF only when:

1. It has a **composite primary key**, and
2. A **non-key attribute** depends only on full composite primary key.

In our relational model:

- The only tables with composite primary keys are linking tables such as **HAS_SKILL, HAS_CARD, OWNS_PONEGLYPH, HAKI, Colour_Scheme, INFO_LEAK, CONTEST**, etc.
- In all these tables all the **non-key attributes** on depends only on the full composite primary key.

Since no relation contains non-key attributes that depend on a subset of a composite key, **no partial dependencies exist anywhere in the schema**.

Therefore, all relations are already in 2NF, and no changes were required.

For 2 NF to 3 NF:

A relation violates 3NF if:

1. It contains a transitive dependency,
i.e., a non-key attribute depends on another non-key attribute.

To verify 3NF:

- We inspected every relation for attributes whose values depend on another non-key attribute.
- All attribute dependencies in the schema are directly on the primary key of their respective relations.
- Weak entities, subclasses, and linking tables contains primary keys and foreign keys and non-key attributes in those relations depends only on primary key.
- No relation contains derived attributes, composite attributes, or attributes that functionally determine others.

Because there are no transitive dependencies in any relation:

All relations are already in 3NF, so no decomposition or modification was required.